

AI Agent Guide

Project: Trading Platform Pro
Application: Trading Cockpit
Version: 1.0
Status: Active

Purpose

This guide defines project-specific operating rules for AI coding agents working on the Trading Platform Pro repository and its primary application, the Trading Cockpit.

It complements:

- `AGENTS.md`
- `Coding\_Standards.md`
- `Development\_Guidelines.md`
- `Testing\_Strategy.md`
- `Git\_Workflow.md`

This document does not replace architecture or capability documentation.

AI agents shall use the current repository implementation and current Markdown documentation as the basis for analysis.

Product Direction

The Trading Cockpit is the primary product.

Current architecture and development decisions shall optimize for concrete Trading Cockpit requirements.

The product is organized around vertical product capabilities such as:

- Market Monitoring
- Decision Center
- Portfolio Management
- Order Management
- Risk Management
- Reporting
- Workspace Management

Shared platform capabilities may evolve when multiple concrete product capabilities require them.

Do not introduce generic platform infrastructure solely for hypothetical future applications.

Prefer:

```text

Trading Cockpit Requirement

↓

Vertical Product Capability

↓

Application and Domain Behaviour

↓

Required Infrastructure

```

over:

```text

Generic Platform Abstraction

↓

Hypothetical Future Consumer

↓

Current Product Adaptation

```

Source of Truth

For implementation analysis:

```text

Current Repository Implementation

→ authoritative for existing code structure

```

For intended architecture and documented behaviour:

```text

Current Markdown Documentation

→ authoritative documentation source

```

Generated files under:

```text

docs/generated/docx/

docs/generated/pdf/

```

are derived artifacts.

Generated DOCX and PDF files shall not be used as documentation source.

When implementation and documentation differ:

1. Identify the discrepancy.
2. Determine the concrete requirement.
3. Determine the intended architecture.
4. Do not silently assume either side is correct.
5. Propose the smallest consistent correction.

Mandatory Workflow

Every implementation task shall follow this sequence:

1. Understand the request.
2. Inspect the current repository implementation.
3. Read affected source files completely where required.
4. Inspect direct callers and dependencies.
5. Read related Markdown documentation.
6. Identify the owning capability and architecture layer.
7. Identify affected state transitions.
8. Identify external side effects.
9. Evaluate PAPER and LIVE impact where relevant.
10. Evaluate duplicate submission risk where relevant.
11. Evaluate retry and acknowledgement semantics where relevant.
12. Evaluate persistence and reconciliation impact where relevant.
13. Design the smallest meaningful change.
14. Present the implementation approach when approval is required.
15. Implement.
16. Add or update automated tests.
17. Update affected Markdown documentation.
18. Run focused tests.
19. Run required regression tests.
20. Run source quality checks.
21. Validate documentation generation where documentation changed.
22. Review the complete diff.
23. Commit.
24. Push.

No required step shall be skipped.

Steps that are genuinely outside the scope of a change may be omitted only after their impact has been evaluated.

Strict Repository Analysis

Never modify code based on assumptions.

Before proposing code changes inspect the current repository.

Do not invent:

- file paths
- modules
- classes
- functions
- method signatures
- configuration fields
- database tables
- database columns
- runtime services
- dependency registrations

Do not rely on previous project versions when the current repository is available.

If a required implementation detail cannot be found, state exactly what is missing.

Do not guess.

File Analysis

Before changing a source file:

- read the relevant implementation
- inspect the complete affected class or workflow
- inspect direct callers
- inspect direct dependencies
- inspect related tests
- inspect related documentation

Reading one isolated function is insufficient when the change affects a multi-step workflow.

For trading-critical changes, follow the complete workflow from Application request to external side effect and resulting business state.

Exact Change Instructions

When providing manual code changes, use exact repository context.

Provide:

1. exact file path
2. exact search location
3. exact replacement or insertion block
4. required verification command

Avoid generic instructions such as:

```
```text
```

```
Update the order service.
```

```
```
```

Prefer:

```
```text
```

File:

```
app/application/orders/order_service.py
```

Search for:

```
async def submit_order(...)
```

Replace the complete method with:

```
...
```

```
```
```

Do not provide speculative patches against files that were not inspected.

Smallest Meaningful Change

Prefer the smallest correct change.

A change should:

- solve one defined requirement
- preserve architecture boundaries
- minimize regression surface
- avoid unrelated refactoring
- remain independently testable

Do not redesign adjacent capabilities without a concrete requirement.

Do not introduce reusable infrastructure solely because reuse may be possible in the future.

Architecture Ownership

Preserve the documented dependency direction:

```text

Presentation

↓

Application

↓

Domain

Infrastructure

↓

Application / Domain Ports

```

Typical ownership:

```text

Domain

→ business rules and invariants

Application

→ use cases and workflow coordination

Infrastructure

→ broker, market data, persistence and technical integrations

Presentation

→ UI and presentation state

```

Code shall be placed according to responsibility, not convenience.

Architecture Boundaries

Always verify where relevant:

- Domain does not depend on Infrastructure.
 - Domain does not depend on Presentation.
 - Application does not depend on Presentation.
 - Presentation does not access concrete broker adapters directly.
 - Provider models remain in Infrastructure.
 - Persistence models remain in Infrastructure.
 - SQLAlchemy types do not leak into Domain.
 - PySide6 types do not leak into Domain.
 - Broker SDK models do not leak into Domain.
- A technically working implementation may still be architecturally invalid.

Architecture Terminology

Do not use architecture labels as substitutes for concrete dependency rules.

Terms such as:

- Clean Architecture
- Domain-Driven Design
- Event-Driven Architecture

may describe architectural influences.

They do not override the documented project boundaries.

The concrete dependency direction, capability ownership and workflow contracts defined in current documentation are authoritative.

Vertical Product Slices

Prefer capability-oriented implementation.

Example:

```
```text
```

Order Management

■■■ Domain behaviour

■■■ Application workflows

■■■ Infrastructure adapters

■■■ Presentation integration

```
```
```

A vertical capability may cross architecture layers while preserving dependency direction.

Do not create horizontal generic services before a concrete capability requires them.

Shared infrastructure should emerge from demonstrated reuse.

Implementation Rules

Prefer:

- small changes
- explicit code
- deterministic behaviour
- strong typing
- focused responsibilities
- explicit state transitions
- explicit side effects
- capability-oriented ownership

Avoid:

- duplicated logic
- speculative code
- unnecessary abstractions
- hidden side effects
- generic dictionaries as stable contracts
- provider models outside Infrastructure
- persistence models outside Infrastructure
- unmanaged background tasks
- silent fallback behaviour

Public and Internal Contracts

Do not preserve a contract solely because it already exists.

Before preserving compatibility identify the real consumer.

Preserve compatibility when:

- an independent consumer exists
- a documented public contract exists
- persisted state requires migration
- a concrete integration depends on the contract

Do not preserve obsolete internal APIs or configuration solely for hypothetical backward compatibility.

Breaking a real contract requires explicit migration analysis.

Trading-Critical Changes

A change is trading-critical when it may affect:

- trading decisions
- order creation
- order validation
- order submission
- order cancellation
- execution processing
- position state
- reconciliation
- financial values
- PAPER or LIVE environment selection

Trading-critical changes require explicit workflow analysis.

Do not refuse such changes solely because they are trading-related when the user request clearly requires them.

Instead apply the required safety analysis.

Trading Safety Analysis

Before changing trading-critical code identify:

1. Current workflow.
2. Intended workflow.
3. Business state before the change.
4. Business state after the change.
5. External side effects.
6. Broker-derived state.
7. Timeout behaviour.
8. Disconnect behaviour.
9. Reconnect behaviour.
10. Duplicate submission risk.
11. Retry behaviour.
12. Persistence impact.
13. Reconciliation impact.
14. PAPER impact.
15. LIVE impact.

If one of these is relevant but cannot be determined from the repository, state the missing information explicitly.

Do not invent behaviour.

Order Side Effects

Order submission and cancellation are external business-critical side effects.

Only explicit Application workflows may request these operations.

Adapters perform technical provider communication.

Do not submit or cancel orders from:

- Domain
- Presentation
- monitoring
- logging
- health checks
- documentation scripts

Reconciliation observation shall not silently submit or cancel orders.

Order Lifecycle

Preserve explicit order lifecycle semantics.

Relevant states may include:

```
```text
```

Submission Requested

Transmitted

Acknowledged

Rejected

Partially Filled

Filled

```
```
```

Do not collapse multiple lifecycle states into:

```
```python
```

```
submitted = True
```

```
```
```

A successful adapter call does not automatically mean broker acknowledgement.

Broker acknowledgement requires broker-derived state.

Duplicate Submission Risk

Every order submission change shall evaluate duplicate submission risk.

Review:

- stable order identity
- submission identity
- repeated callbacks
- repeated broker messages
- timeout behaviour
- disconnect behaviour
- reconnect behaviour
- application restart behaviour
- recovery behaviour

Unknown external order state shall not automatically trigger resubmission.

Do not blindly repeat order submission after:

- timeout
- disconnect
- reconnect
- unknown acknowledgement state

Duplicate-prevention rules belong to the owning Application workflow.

Retry Behaviour

Retries shall be explicit.

Before adding retry evaluate:

- idempotency
- external side effects
- duplicate execution risk
- provider semantics
- workflow ownership

Automatic retry may be appropriate for selected read operations.

Automatic order submission retry is prohibited unless explicitly defined and approved by the owning order workflow.

Do not place generic retry decorators around trading-critical side-effect operations.

Broker Reconnection

Broker reconnection and order submission are separate concerns.

A reconnect workflow shall not automatically repeat:

- order submission
- order cancellation
- trading decisions

unless an explicit Application recovery workflow owns and validates the behaviour.

Infrastructure connection recovery shall not independently decide business recovery actions.

Reconciliation

Reconciliation compares local and external state.

Reconciliation observation may:

- load local state
- load broker state
- detect discrepancies
- classify discrepancies
- expose action-required state

Reconciliation observation shall not silently repair business state.

Repair requires an explicit Application workflow.

Monitoring and logging are not reconciliation repair owners.

PAPER and LIVE

Trading-related changes shall evaluate execution environment impact.

Use explicit environments:

```
```text
```

PAPER

LIVE

```
```
```

Rules:

- PAPER remains PAPER.
 - LIVE requires explicit selection.
 - No fallback may activate LIVE.
 - CI shall not require LIVE trading.
 - Automated tests shall not submit LIVE orders.
 - Environment context shall remain visible where operationally relevant.
- Do not infer execution environment from broker port or account naming conventions.

Financial Data

Financial values require explicit semantics.

Use `Decimal` where decimal financial precision is required.

Unavailable financial data shall remain unavailable.

Do not silently convert missing values to:

```
```python
```

0

0.0

Decimal("0")

```
```
```

when zero has valid business meaning.

Missing data and zero are different business states.

AsyncIO

Async code shall:

- not block the event loop
- use explicit task ownership
- support controlled cancellation
- use explicit timeout behaviour where appropriate
- preserve exception context

Avoid unmanaged:

```
```python
asyncio.create_task(...)
```
```

for long-running application services.

A long-running task requires:

- owner
- identity
- startup behaviour
- cancellation behaviour
- failure handling
- shutdown behaviour

Presentation

Presentation code owns:

- rendering
- user interaction
- presentation state
- Application command invocation

Widgets shall not:

- construct broker adapters
- access SQLAlchemy sessions
- submit orders directly
- implement trading rules

Trading actions shall invoke explicit Application workflows.

Monitoring and Logging

Monitoring and logging observe behaviour.

They shall not:

- submit orders
- cancel orders
- retry order submission
- reconnect brokers independently
- repair Domain state
- resolve reconciliation discrepancies

Monitoring may expose operational state.

Logging may record deterministic events.

Neither is authoritative business state.

Testing Requirements

Every change requires appropriate verification.

Testing depth depends on risk.

Potential test categories:

- Unit Tests

- Integration Tests
- Trading Workflow Regression Tests
- Runtime and System Tests

Trading-critical changes require explicit regression scenarios where practical.
Tests shall not require LIVE trading.

Test Doubles

Use controlled architecture boundaries.

Potential test doubles include:

- fake repositories
- fake broker ports
- fake market data ports
- fake clocks
- fake event dispatchers
- controlled runtime services

Do not mock the business rule being tested.

Example:

When testing duplicate submission prevention, exercise the real duplicate-prevention rule and fake the broker boundary.

Deterministic Tests

Tests shall not depend on uncontrolled:

- wall-clock time
- network state
- LIVE broker services
- current market state
- random data
- developer-local configuration

Use controlled clocks, fakes and deterministic fixtures where required.

Avoid arbitrary sleeps.

Required Trading Regression Scenarios

Where the affected workflow is implemented, evaluate tests for:

- order validation
- duplicate submission prevention
- broker acknowledgement
- broker rejection
- timeout
- disconnect
- reconnect
- partial fill
- fill
- cancellation
- repeated broker messages
- position lifecycle
- reconciliation discrepancy

Only relevant scenarios need to be added.

Do not create hypothetical tests for unimplemented capabilities.

Documentation

Documentation is part of implementation.

Markdown under:

```
```text
docs/
```
```

is the documentation source of truth.

When a change affects documented:

- architecture
- behaviour
- API
- configuration
- runtime lifecycle
- workflow
- operational state

update the affected Markdown documentation before final verification.

Documentation Generation

When Markdown documentation changes run:

```
```bash
python scripts/generate_docs.py
```
```

The command shall complete successfully.

Generated DOCX and PDF files shall not be re-read for content validation.

Do not manually edit generated documentation.

Source Quality

Required source quality checks include:

```
```bash
ruff check .
ruff format --check .
```
```

Use focused tests before broad regression execution.

Run the quality gates relevant to the change before commit.

Complete Diff Review

Before commit review the complete diff.

Verify:

- only intended files changed
- no unrelated refactoring exists
- no debug code remains
- no temporary code remains
- no secrets were added
- tests match implementation
- documentation matches implementation

For trading-critical changes additionally verify:

- external side effects
- state transitions
- duplicate submission risk
- retry behaviour
- PAPER and LIVE impact
- reconciliation impact

Communication

Responses shall be:

- concise
- factual
- implementation-oriented
- explicit about uncertainty

Avoid:

- repetition
- speculation
- invented repository details
- generic architecture lectures
- unnecessary alternatives after a direction is already established

If information is missing, state exactly what is missing.

If a repository fact has not been verified, do not present it as verified.

Approval

Wait for approval when:

- the user explicitly requests approval before implementation
- multiple materially different architecture directions exist
- a trading-critical behaviour change is not clearly defined
- a destructive migration is required
- a real public contract would be broken without an agreed migration

Do not request redundant approval for every routine implementation step when the requested change is already explicit.

Commit and Push

Commit only after:

- implementation is complete
- relevant tests pass
- required regression tests pass
- source quality checks pass
- documentation is synchronized
- documentation generation passes where required
- complete diff is reviewed

Push after local verification and commit.

A successful push does not replace CI validation.

Definition of Success

A task is complete only when:

- the requested requirement is implemented
- architecture boundaries remain consistent
- capability ownership remains clear
- side effects are explicit
- state transitions are deterministic
- relevant tests pass
- trading regression scenarios pass where required
- documentation is synchronized
- documentation generation passes where required
- source quality checks pass
- complete diff is reviewed
- changes are ready for repository integration

For the standard development workflow, completion also includes commit and push.

A task with known unresolved trading-critical behaviour is not complete.

AI Agent Review Checklist

Before completing a task verify:

- current repository inspected
- affected files inspected
- direct callers inspected
- direct dependencies inspected
- related tests inspected
- related Markdown documentation inspected
- owning capability identified
- architecture layer identified
- smallest meaningful change selected
- no repository details invented
- side effects identified
- state transitions identified
- financial availability semantics preserved
- async task ownership evaluated
- timeout behaviour evaluated
- cancellation evaluated
- duplicate submission risk evaluated where relevant
- retry behaviour evaluated where relevant
- broker acknowledgement semantics evaluated where relevant
- reconnect behaviour evaluated where relevant
- reconciliation impact evaluated where relevant
- PAPER and LIVE impact evaluated where relevant
- tests added or updated
- focused tests passed
- required regression tests passed
- Ruff passed
- formatting verification passed
- documentation synchronized
- documentation generation passed where required
- complete diff reviewed
- no secrets exposed
- commit completed where required
- push completed where required

Related Documents

- Product_Vision.md
- Product_Roadmap.md
- Project_Overview.md
- Architecture.md
- Domain_Model.md
- Infrastructure.md
- Technical_Specifications.md
- API_Guidelines.md
- Coding_Standards.md
- Development_Guidelines.md
- Git_Workflow.md
- Testing_Strategy.md
- Configuration.md
- Runtime.md
- Logging.md
- Monitoring.md
- CI_CD.md

- AGENTS.md